RENTipid Insurance Module

Full Technical, Security, Data, Operational and Closure Documentation

RENTipid Engineering

12 August 2026

Document control

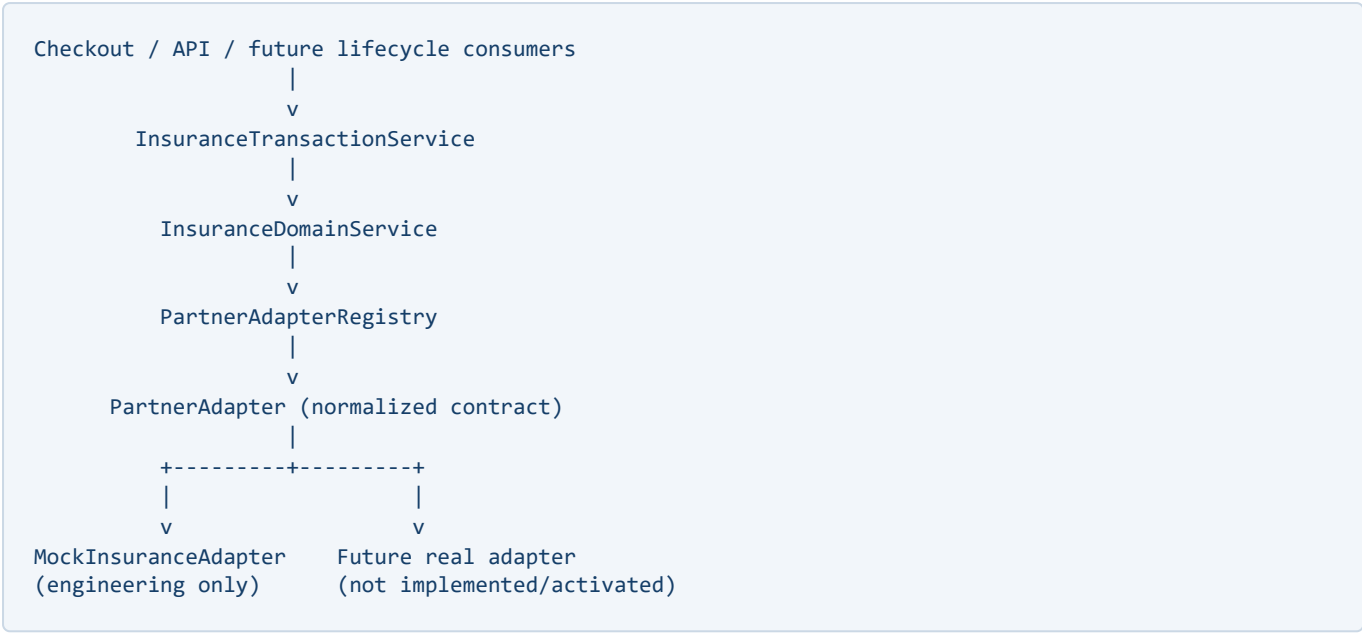
Field	Value
Module	TRU-01 Insurance
Document status	FINAL
Project disposition	CLOSED / FROZEN / SAFELY SHELVED
Live insurance activation	DISABLED / NOT ACTIVATED
Production deployment performed	NO
Production database action performed	NO
Frozen Technical Foundation baseline	2ff068991950de64e3bf0931ed76a5650217dbe2
Transaction Block source baseline	6e22684907487d961146661547f29badbcd59dc9
Foundation freeze ID	FRZ-INS-S1-2026-001
Transaction change record	CR-2026-INS-001
Source registries	docs/insurance/implementation/R1 through R15
Classification	Internal engineering and operations documentation

> **Authoritative disposition.** The owner has declared TRU-01 Insurance fully closed, frozen and safely shelved. Shelved means the module is retained as a controlled, non-live engineering baseline. It does not mean that a regulated insurer, product wording, premium, coverage, claims promise, Production deployment or live policy issuance has been approved or activated.

1. Executive summary

RENTipid Insurance is a provider-neutral insurance orchestration module built around a normalized domain boundary. It supports deterministic engineering eligibility and offers, optional checkout presentation, explicit consent, insurance selection and order records, payment-dependency gating, policy lifecycle foundations, idempotency and verified webhook processing.

The architecture deliberately separates RENTipid business logic from insurer implementations:



The architecture is fail-closed. Insurance, Mock usage and live issuance require explicit configuration. The kill switch defaults active when its configuration is absent. Ordinary rental checkout can continue without Insurance when the optional subsystem is unavailable.

1.1 Final safe-state summary

Control	Final state
Insurance feature	Disabled unless explicitly enabled
Live policy issuance	Disabled
Mock adapter	Explicit opt-in only; non-live
Kill switch	Safe default active
Real insurer credentials	Not required or stored
Real insurer requests in accepted evidence	0
Real payment movement in accepted evidence	0
Production actions	None
Provider-specific branches in normalized core	None found
Frozen foundation	Closed and immutable
Transaction implementation	Locally accepted and committed
Shelf posture	Non-live, controlled, change-controlled

2. Scope

2.1 Implemented technical scope

- normalized identifiers, money, eligibility, offer, consent, policy, claim, webhook, reconciliation and capability contracts;
- PartnerAdapter interface with eleven normalized capabilities;
- deterministic PartnerAdapterRegistry and MockInsuranceAdapter;
- fail-closed InsuranceConfig and kill-switch boundary;
- InsuranceDomainService and InsuranceTransactionService;
- eligibility with machine-safe reason codes;
- offers with integer minor-unit money;
- optional checkout UI and bounded checkout integration;
- explicit, non-preselected affirmative consent;
- persisted selection and order lifecycles;
- payment-dependency abstraction;
- idempotent selection, order creation, issuance and policy persistence;
- verified, replay-resistant and duplicate-safe webhook foundation;
- normalized audit events;
- authenticated API and ownership boundaries;
- additive Prisma schema and migrations;
- deterministic local-only Mock partner/product sync;
- focused automated validation and local acceptance.

2.2 Intentionally excluded or non-live scope

- real insurer adapter and credentials;
- approved wording, exclusions, premiums or deductibles;
- regulatory or commercial insurer approval;
- live underwriting and policy issuance;
- live partner webhook activation;
- real insurance payment, settlement or payout;
- cancellation/refund execution;
- full claim and evidence lifecycle;
- Insurance ledger and finance reconciliation;
- database-backed Super Admin kill-switch UI;
- Production deployment or Production database operations.

3. Architectural principles

3.1 Provider neutrality

All consumers call normalized RENTipid contracts. Partner-specific authentication, endpoints, headers, request/response fields, errors, webhook formats and status mappings belong only inside adapters. Booking, Checkout, Payment, Claims, Finance and UI logic must never branch on a particular insurer.

The accepted targeted scan of src/lib/insurance/transaction found no prohibited production-provider branching.

3.2 Fail-closed operation

New business operations require Insurance explicitly enabled, kill switch inactive, an adapter configured and registered, Mock explicitly enabled when selected, and live issuance explicitly enabled for any non-Mock adapter. Missing or invalid configuration produces safe domain errors. The module never silently selects Mock or a production adapter.

3.3 Exact money

Money uses an integer amountMinor plus an uppercase three-letter currency. Floating-point premium logic at domain boundaries is prohibited. The existing rental payment amount is not modified by the optional Insurance UI.

3.4 Optional checkout

Insurance is never preselected. Insurance failure does not intentionally block ordinary rental checkout. Premium is displayed separately and is not silently added to the rental amount.

3.5 Reuse

The module reuses existing User, Booking, authentication, audit, Payment, Prisma and migration infrastructure. It does not duplicate Booking, Payment, Finance, Audit or RBAC systems.

4. Component catalog

Component	Responsibility
InsuranceConfig	Parse fail-closed environment configuration
ConfigInsuranceKillSwitch	Expose kill-switch state
PartnerAdapter	Normalize partner capabilities
PartnerAdapterRegistry	Deterministic registration and resolution
MockInsuranceAdapter	Deterministic engineering scenarios
InsuranceDomainService	Provider-neutral capability orchestration
InsuranceTransactionService	Checkout, consent, order, issuance and webhooks
InsuranceTransactionRepository	Persistent transaction contract
PrismaInsuranceTransactionRepository	Prisma persistence
InsurancePaymentDependency	Isolate Insurance from Payment
DeferredInsurancePaymentDependency	Report pending without fake success
FixtureInsurancePaymentDependency	Deterministic test fixture
InsuranceCheckoutOption	Optional non-preselected checkout UI
processOptionalInsuranceCheckout	Non-blocking checkout boundary
RentipidInsuranceAuditSink	Existing audit infrastructure adapter

5. Partner adapter contract

Capability	Purpose
checkEligibility	Return normalized eligibility
getOffers	Return normalized offers
createOrder	Create order/policy result
getPolicy	Retrieve policy
cancelPolicy	Request supported cancellation
createClaim	Submit normalized claim
getClaim	Retrieve normalized claim
verifyWebhook	Verify and normalize event
reconcile	Produce reconciliation result
getCapabilities	Report support
healthCheck	Report safe availability

The Mock adapter implements all capabilities with stable fixtures. Mock IDs are clearly fake and cannot be mistaken for real insurer references.

6. Eligibility and offers

Eligibility consumes request, user, Booking and Listing identifiers, general Listing category, rental value, currency and rental dates. Inputs are validated for non-empty IDs, safe integer money, three-letter currency and a valid rental period.

Results are ELIGIBLE, INELIGIBLE or TEMPORARILY_UNAVAILABLE with machine-safe reason codes. Adapter failure becomes temporary unavailability for optional checkout.

Offers contain offer, product and partner identifiers, currency, premium minor units, coverage reference/dates, expiry, disclosure version, status and Mock marker. The Mock fixture uses PHP and mock-terms-not-insurance-v1. These are engineering values, not approved insurance terms.

7. Checkout and consent

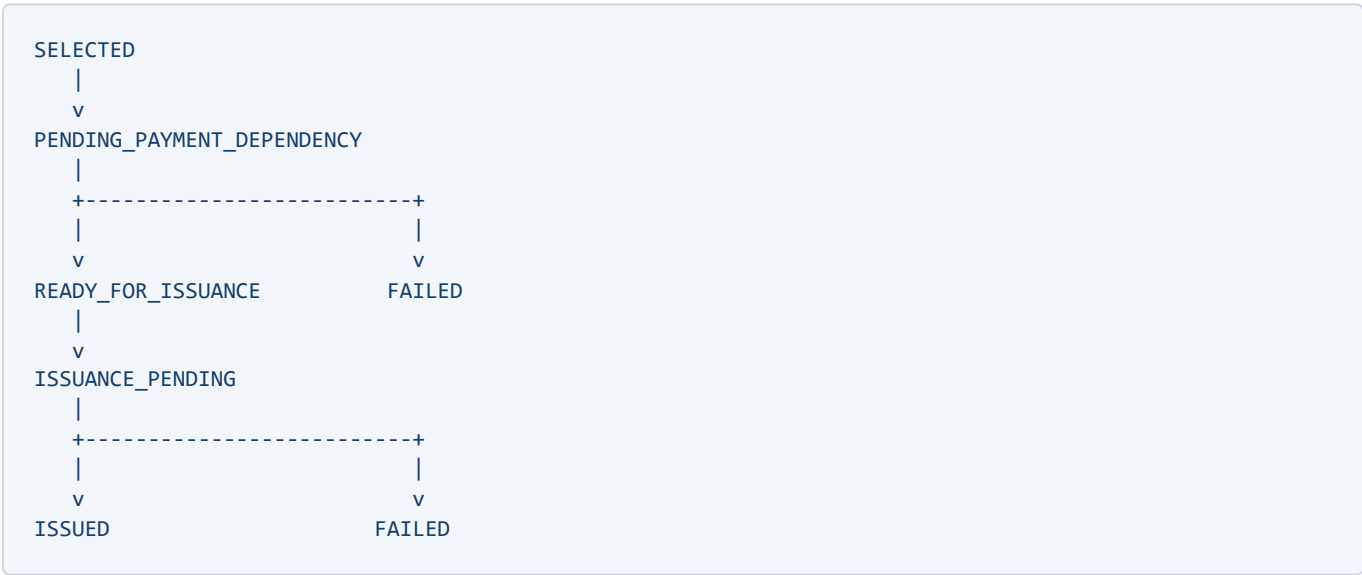
The checkout component loads an authenticated offer, shows an unchecked optional selection, reveals a separate consent checkbox after selection, identifies Mock data as non-live, displays premium separately and leaves the existing rental payment unchanged.

The server authenticates the renter, verifies Booking ownership, rebuilds canonical context, retrieves the authoritative offer, verifies expiry, compares disclosure/premium/currency and records the selection idempotently.

Consent evidence includes user and Booking, offer/partner/product references, disclosure version, presented premium, currency, coverage dates, expiry, server-side consent time, idempotency key and request hash. The database requires consent_accepted to be true.

8. Order and policy lifecycle

Order states:



The default payment dependency reports PENDING and never treats a rental transaction as payment of an Insurance premium. Issuance needs authorized or settled status with an exact amount/currency match.

Non-Mock issuance is blocked while live issuance is disabled. Mock issuance is engineering-only. Successful policy persistence happens once; failures produce safe state and audit.

Policy states are PENDING, ACTIVE, CANCELLED, EXPIRED and FAILED. Cancellation/refund execution remains outside the shelved block.

9. Idempotency and concurrency

Keys are deterministic SHA-256 digests scoped by operation, principal, Booking and request. Request hashes bind semantic payloads.

Mutation	Protection
Selection	Unique key and one selection per Booking
Order	Unique key and one order per selection/Booking
Issuance	Unique issuance key and request hash
Policy	Unique Booking, order and idempotency constraints
Webhook	Unique partner/event ID and body hash

Identical retries return stable results. Same-scope different-payload replays fail. Prisma unique-conflict recovery compares hashes before returning an existing record.

10. Webhook processing

Sequence:

1. match route partner to configured adapter;
2. stop when kill switch is active;
3. verify signature and body through the adapter;
4. require event ID, type, time and body hash;
5. reject outside the five-minute replay window;
6. detect existing partner/event;
7. return duplicate for the same body;
8. reject conflicting body hash;
9. insert a pending event;
10. map known policy events/status;
11. update matching policy when applicable;
12. mark processed or ignored;
13. write normalized audit.

Unknown verified events are safely ignored. Raw partner errors and bodies are not exposed to users.

11. API surface

Route	Method	Boundary
/api/insurance/offers	POST	Authenticated renter; Booking ownership
/api/insurance/select	POST	Explicit consent and authoritative offer
/api/insurance/orders	POST	Owner-scoped idempotent creation
/api/insurance/orders/[id]/issuance	POST	Payment and issuance gates
/api/insurance/policies/[id]	GET	Renter/provider/admin ownership
/api/webhooks/insurance/[partner]	POST	Signature and replay validation

Errors use safe codes and generic messages. Credentials and raw provider errors are never returned.

12. Data model

Foundation models:

- InsurancePartner: adapter metadata/capabilities, never credentials;
- InsuranceProduct: normalized product/configuration;
- InsuranceOffer: optional booking-linked offer;
- InsurancePolicy: policy lifecycle;
- InsuranceClaim: claim foundation;
- InsuranceWebhookEvent: verified event and replay record.

InsuranceSelection stores immutable consent and offer evidence. It has unique Booking/idempotency constraints, a true-consent check, non-negative minor units, allowed status check and restricted User/Booking foreign keys.

InsuranceOrder stores payment/issuance lifecycle. It has unique selection, Booking, idempotency and issuance keys; allowed state checks; and restricted User, Booking and selection relations.

InsurancePolicy has an additive optional unique insurance_order_id.

12.1 Migrations

Migration	Scope	State
20260812000000_add_insurance_foundation	Six foundation models	Accepted/frozen
20260812010000_add_insurance_transaction_block	Selection/order/policy relation	Applied locally

Both are additive. No reset, table drop, history fabrication or Production database action occurred.

13. Required data and configuration

The guarded local Mock sync converges to one mock partner and one MOCK-FOUNDATION product. Both are LOCAL, MOCK_LOCAL_ONLY, non-production and not approved insurance. It seeds no offers, selections, orders, policies, claims or webhook events.

Safe shelf configuration:

Variable	Value
INSURANCE_ENABLED	false
INSURANCE_LIVE_ISSUANCE_ENABLED	false
INSURANCE_MOCK_ENABLED	false
INSURANCE_KILL_SWITCH	true
INSURANCE_ADAPTER	Unset

14. RBAC and ownership

Actor	Boundary
Guest	No access
Renter	Own offers, selection, order and policy
Provider	Policies attached to owned Booking/listing
Admin	Existing administrative RBAC
Finance Admin	Finance/reconciliation only
Compliance Admin	Compliance/claims only
Super Admin	Full control and future database kill switch
Partner service	Future authenticated webhook/sync only

Routes reuse existing sessions and Booking ownership. Services do not invent ownership.

15. Audit

Events include eligibility checked, offer presented, selection, consent, order created, issuance requested, policy issued/failed and webhook received/rejected.

Metadata is minimal: IDs, normalized states, safe reason codes, product, currency/premium and adapter ID. It excludes credentials and raw partner payloads. Required audit failure fails the protected operation safely.

16. Security controls

Risk	Control
Unauthenticated use	Session checks
IDOR	Booking/order/selection ownership
Silent purchase	Unchecked selection and separate consent
Premium tampering	Server offer re-fetch and exact comparison
Money discrepancy	Integer minor units
Double issuance	Idempotency, hashes and unique constraints
Webhook spoofing	Adapter verification
Cross-adapter event	Configured adapter identity
Replay	Time window and unique event
Conflicting duplicate	Body-hash rejection
Provider leakage	Normalized errors
Accidental live issuance	Disabled by default
Mock fallback	Explicit Mock enablement
Emergency	Kill switch before mutation
Secret exposure	No credentials in normalized models/logs

17. Privacy

Only minimum transaction data crosses the adapter boundary: general category, dates, value, opaque identifiers and consent context. Offers are low sensitivity; consent is immutable; policies are confidential; claims/evidence are sensitive and deferred; webhook bodies become normalized metadata/hash; audit is restricted.

Any future live adapter requires lawful-basis, minimization, retention, encryption, cross-border and data-processing review.

18. Finance boundary

The module moves no real money. Future activation must independently trace Insurance quote, payment dependency, order/policy, premium hold/settlement, refund, claim adjustment and reconciliation. Insurance must remain separate from rental amount, security deposit, provider payout and marketplace fee.

19. Product governance

Code	Intent	Activation
RIP-AD	Accidental damage	NOT ACTIVATED
RIP-THEFT	Theft	NOT ACTIVATED
RIP-NR	Non-return	NOT ACTIVATED
RIP-TRANSIT	Transit loss/damage	NOT ACTIVATED
RIP-PA	Personal accident	NOT ACTIVATED
RIP-TPL	Third-party liability	NOT ACTIVATED
RIP-MOTOR	Motor-specific	NOT ACTIVATED
RIP-HOME	Home/property	NOT ACTIVATED
RIP-BIZ	Business continuity	NOT ACTIVATED
RIP-CANCEL	Cancellation	NOT ACTIVATED

These codes describe software intent only. Wording, exclusions, deductible, premium, insurer and regulatory approvals remain external.

20. Validation and evidence

20.1 Foundation Slice 1

Gate	Evidence	Result
CODE COMPLETE	EVD-INS-S1-GATE1	PASS
LOCAL FUNCTIONAL	EVD-INS-S1-GATE2	PASS
LOCAL DATABASE MIGRATED	EVD-INS-S1-GATE3	PASS
LOCAL DATA	EVD-INS-S1-GATE4	PASS
LOCAL ACCEPTANCE	EVD-INS-S1-GATE5	PASS
PREVIEW MIGRATED	EVD-INS-S1-GATE6	PASS
PREVIEW ACCEPTANCE	EVD-INS-S1-GATE7	PASS
PRODUCTION-READY	EVD-INS-S1-GATE8	PASS
CLOSED / FROZEN	EVD-INS-S1-GATE9	PASS

Foundation tests: 17 passed, 0 failed.

20.2 Transaction Block

Gate	Evidence at final run	Result
CODE COMPLETE	EVD-INS-TX-GATE1	PASS
LOCAL FUNCTIONAL	EVD-INS-TX-GATE2	PASS
LOCAL DATABASE MIGRATED	EVD-INS-TX-GATE3	PASS
LOCAL DATA SYNCED	EVD-INS-TX-GATE4	PASS
LOCAL ACCEPTANCE	EVD-INS-TX-GATE5	PASS
Preview promotion	EVD-INS-TX-GATE6	Not completed in last run

Transaction tests: 14 passed, 0 failed. Prisma validate/generate passed on 6.19.3. Lint, provider-neutrality and Transaction TypeScript passed. Four unrelated existing diagnostics in src/lib/auth.ts were not modified.

The owner's later CLOSED / FROZEN / SAFELY SHELVED declaration is an administrative project disposition. It freezes a safe non-live state and does not convert incomplete Transaction Preview promotion into Production acceptance.

21. Deployment and recovery

Accepted foundation Preview:

- deployment dpl_CAZtitCnmuRL2hdf9hEjfT5gxukS;
- source 2ff068991950de64e3bf0931ed76a5650217dbe2;
- Preview / READY;
- Insurance disabled, live issuance disabled, Mock disabled, kill switch active;
- Production action none.

Transaction record:

- source 6e22684907487d961146661547f29badbcd59dc9;
- branch push succeeded;
- local baseline 39 migrations, current;
- Preview Insurance booleans safe;
- final Preview database/auth config was not operational in the last run;
- no Transaction Preview database write;
- no Production action.

Safe recovery:

1. Keep Insurance/live/Mock disabled and kill switch active.
2. Do not add real credentials while shelved.
3. Preserve commits, migrations and evidence.
4. Create change control before reactivation.
5. Re-establish dedicated Preview database/auth.
6. Resume only at the first unproved gate.
7. Require wording, partner, security, finance and Production approvals.

22. Shelving and reactivation

While shelved: no feature work, migration edits, seed changes, partner onboarding, credentials, deployment, live issuance, real payment or claims promise.

Reactivation requires business owner/reason, affected scope, partner/product approval, wording approval, privacy/security review, Booking/Payment contract, migration/rollback analysis, focused regression, Local/Preview acceptance and explicit Production authorization.

23. Known limitations

- no real partner adapter or approved product;
- database Super Admin kill switch deferred;

- live Booking/payment issuance deferred;
- cancellation/refund deferred;
- claims/evidence deferred;
- Insurance ledger/reconciliation deferred;
- no Production operational history;
- Transaction Preview migration/acceptance was not completed before shelving.

24. Change control

Foundation is frozen as FRZ-INS-S1-2026-001. Transaction work is CR-2026-INS-001. Future changes must identify baseline/reason/scope, list contracts/routes/models, assess data/security/privacy/finance impact, run affected promotion gates, record a new commit/freeze and preserve old evidence.

25. Source inventory

Core:

- src/lib/insurance/types.ts
- src/lib/insurance/PartnerAdapter.ts
- src/lib/insurance/PartnerAdapterRegistry.ts
- src/lib/insurance/InsuranceConfig.ts
- src/lib/insurance/InsuranceDomainService.ts
- src/lib/insurance/adapters/MockInsuranceAdapter.ts

Transaction:

- src/lib/insurance/transaction/InsuranceTransactionService.ts
- src/lib/insurance/transaction/PrismaInsuranceTransactionRepository.ts
- src/lib/insurance/transaction/repository.ts
- src/lib/insurance/transaction/types.ts
- src/lib/insurance/transaction/idempotency.ts
- src/lib/insurance/transaction/payment-dependency.ts
- src/lib/insurance/transaction/booking-context.ts
- src/lib/insurance/transaction/optional-checkout.ts
- src/lib/insurance/transaction/runtime.ts
- src/lib/insurance/transaction/http.ts

Routes/UI:

- src/app/api/insurance/offers/route.ts
- src/app/api/insurance/select/route.ts
- src/app/api/insurance/orders/route.ts
- src/app/api/insurance/orders/[id]/issuance/route.ts
- src/app/api/insurance/policies/[id]/route.ts

- src/app/api/webhooks/insurance/[partner]/route.ts
- src/app/checkout/[bookingId]/InsuranceCheckoutOption.tsx

Data/tests:

- prisma/schema.prisma
- prisma/migrations/20260812000000_add_insurance_foundation/migration.sql
- prisma/migrations/20260812010000_add_insurance_transaction_block/migration.sql
- scripts/sync-insurance-local-mock-catalog.ts
- tests/insurance/foundation.spec.ts
- tests/insurance/transaction-block.spec.tsx

Documentation: R1 through R15 under docs/insurance/implementation.

26. Glossary

Term	Meaning
Adapter	Provider implementation behind normalized contract
Affirmative consent	Explicit auditable opt-in
Fail closed	Disable/reject when safety state is absent
Idempotency	Stable retry; conflict rejection
Live issuance	Real regulated policy creation
Mock	Deterministic engineering behavior
Normalized	RENTipid provider-neutral contract
Production-ready	Technical decision, not deploy permission
Shelved	Disabled, retained and change-controlled
Webhook replay	Reuse of old/duplicate event

27. Final certification

TRU-01 Insurance is documented as **CLOSED / FROZEN / SAFELY SHELVED** by owner authority. Accepted implementation and evidence are preserved. The module remains disabled and non-live. Nothing here represents insurer approval, wording approval, regulatory authorization, Production deployment, Production database activity or live insurance availability.

Future work must begin with targeted change control and the first unproved gate. No frozen baseline may be silently reopened.